



UNIVERSITAS
GADJAH MADA

Fungsi

demuji@ugm.ac.id

ugm.ac.id

LOCALLY ROOTED, GLOBALLY RESPECTED



Fungsi



Apa itu Fungsi?

- Blok kode terstruktur (independent) untuk tugas spesifik.
- Tujuan: Memecah program menjadi bagian-bagian kecil yang modular & reusable.
- Sintaks: nama fungsi, parameter (opsional), dan isi (body) fungsi.
- Pemanggilan: nama(argumen)
- Return Value: Fungsi mengembalikan nilai menggunakan kata kunci return

Manfaat

- Modularity
- Reusability
- Menghindari duplikasi code
- Mempermudah pemeliharaan

Anatomi Fungsi

- Header: identifier (nama), parameter
- Body: Implementasi logic fungsi
- Return: Kembalian hasil

```
+-----+
|      Nama Fungsi      |
+-----+
|      Parameter         |
+-----+
|      Body              |
|                        |
|                        |
+-----+
|      Return            |
+-----+
```

Fungsi di C++

```
// Fungsi dengan return
float funcHitungBMI(float berat, float tinggi) {
    return berat / (tinggi * tinggi);
}

// Void function
void funcSalam(string username) {
    cout << "Welcome, " << username;
}
```

Khas:

- Prototyping
- Overloading
- Pass by reference (spesifik)

Prototyping

- Deklarasi fungsi header saja
- Identitas fungsi dikenali di main
- Kompilasi dan deteksi error lebih cepat

```
// Prototipe
float konversiSuhu(float celsius);

int main() {
    cout << konversiSuhu(37.5);
}

// Implementasi
float konversiSuhu(float celsius) {
    return (celsius * 9/5) + 32;
}
```

Overloaded

- Mendefinisikan beberapa fungsi dengan identitas sama
- Pembeda: karakteristik parameter
 - jumlah
 - tipe

```
void cetak(int x){ cout << "Integer: " << x; }  
void cetak(int x, int y){cout << "X:" << x << "Y:" << y;}  
void cetak(double x){ cout << "Double: " << x; }  
void cetak(string s){ cout << "String: " << s; }
```


Fungsi di Python

```
# Fungsi dengan return
def func_hitung_BMI(berat, tinggi):
    return berat / (tinggi ** 2)

# Void function
def func_salam(username):
    print(f"Welcome, {username}")
```

Khas:

- **Dynamic typing**
- Multi return

Multi Return

```
def operasi(a, b):  
    jumlah = a + b  
    selisih = a - b  
    return jumlah, selisih
```

```
x, y = operasi(10, 3)  
print("Jumlah:", x)      # Output: Jumlah: 13  
print("Selisih:", y)     # Output: Selisih: 7
```

```
hasil = operasi(10, 3)  
print(hasil[0])  # Jumlah: 13  
print(hasil[1])  # Selisih: 7
```

Ekspresi Lambda: Python

- Fungsi singkat tanpa nama, hanya 1 ekspresi
- sintaks: `lambda arg1, arg2, ... : ekspresi`
- Bisa sebagai argumen

```
tambah = lambda a, b: a + b  
print(tambah(3, 5))    # Output: 8
```

```
data = [(1, 'apel'), (3, 'pisang'), (2, 'jeruk')]  
sorted_data = sorted(data, key=lambda x: x[1])  
print(sorted_data)  
# Output: [(1, 'apel'), (2, 'jeruk'), (3, 'pisang')]
```

Ekspresi Lambda: C++

- Anonymous, lebih eksplisit
- Sintaks:

```
[capture] (parameter) -> return_type {  
    // body  
};
```

```
int main() {  
    int z = 2;  
    auto tambah = [z](int a, int b) {  
        return a + b + z;  
    };  
  
    cout << tambah(3, 5) << endl; // Output: 10  
    cout << [](int x, int y) { return x * y; }(4, 6); // Output: 24  
    return 0;  
}
```

Parameter

- C++: deklarasi tipe setiap parameter
- Opsi Default value
 - Nilai default jika argument kosong
 - Strategi untuk parameter opsional

```
void salam(string nama = "User") {  
    cout << "Hai, " << nama;  
}
```

```
def greet(nama="User") :  
    print(f"Hai, {nama}")
```

Variable Scope

- Jenis variabel: lokal, global
- Lokal: hanya dikenal di dalam fungsi
- Global: dikenal diseluruh program

Pass by Value, by Reference

- by Value: nilai yang diteruskan
- by Reference: buffer variabel yang diteruskan (wadah) ; mutable object di python

```
void ubahNilai(int &x, int y) {  
    x = 100;  
    y = 200;  
}  
  
int main() {  
    int a = 10; b = 20;  
    ubahNilai(a, b);  
    cout<<"a: "<<a<<" b: "<<b <<endl;  
    // Output: a: 100 b: 20  
    return 0;  
}
```

```
def ubah(lst):  
    lst.append(100)  
  
angka = [1, 2, 3]  
ubah(angka)  
print(angka)  
# Output: [1, 2, 3, 100]
```

QnA